CSP-J 2021 (Junior)

2021-10-23, 8:30–12:00 (3.5 hours)

This file is tailored to QOJ: every problem reads from standard input and writes to standard output (the original paper used the files `<code>.in` / `<code>.out` named after each problem).
Translated from the official Chinese statements of the second round of CSP-J 2021 (CCF non-professional software capability certification, junior level).

Problem 1: Distributing Candy (分糖果, candy) QOJ 18075

Time limit: 1 s. Memory limit: 512 MB. Total points: 100.

Background

The children of Red Sun Kindergarten are starting to distribute candy!

Description

Red Sun Kindergarten has n children, and you are one of them. It is guaranteed that $n \geq 2$.

One day you discover infinitely many pieces of candy in the back garden of the kindergarten, and you plan to take some candy back to distribute among the children of the kindergarten.

Since you are just an ordinary kindergarten child, your strength is limited, so you can take back at most R pieces of candy.

But taking too few would not be enough to distribute, so you must take back at least L pieces of candy. It is guaranteed that $n \leq L \leq R$.

In other words, if you take k pieces of candy, then you must guarantee that $L \leq k \leq R$.

If you take k pieces of candy, you will put these k pieces into a basket and ask everyone to distribute the candy according to the following scheme: as long as the basket contains no fewer than n pieces of candy, all n children of the kindergarten (including yourself) each take exactly one piece of candy from the basket, until the number of pieces in the basket is fewer than n. At this moment all the candy remaining in the basket belongs to you — these pieces are your reward for carrying the candy.

As a high-quality kindergarten child, you want the number of pieces that are your reward for carrying the candy (not the total number of pieces you end up with!) to be as large as possible; therefore you need to write a program that reads n, L, R in order and outputs the maximum number of pieces of candy you can obtain as your reward for carrying the candy.

Input

The input consists of one line containing three positive integers n, L, R, denoting the number of children, the lower bound on the number of pieces of candy, and the upper bound, respectively.

Output

Output one line containing one integer: the maximum number of pieces of candy you can obtain as your reward for carrying the candy.

Sample 1 input

```
7 16 23
```

Sample 1 output

```
6
```

Explanation of sample 1

Take $k = 20$ pieces of candy and put them into the basket.

The basket now contains $20 \geq n = 7$ pieces, so every child gets one piece;

the basket now contains $13 \geq n = 7$ pieces, so every child gets one piece;

the basket now contains $6 < n = 7$ pieces, so these 6 pieces are your reward for carrying the candy.

It is easy to see that the number of pieces you obtain as your reward for carrying the candy cannot exceed 6 (otherwise the number of pieces left in the basket would still be no fewer than n, and every child would have to take another piece), so the answer is 6.

Sample 2 input

```
10 14 18
```

Sample 2 output

```
8
```

Explanation of sample 2

It is easy to see that when the number k of pieces you take satisfies $14 = L \leq k \leq R = 18$, after every child gets one piece the remaining $k - 10$ pieces are always the number of pieces that are your reward for carrying the candy, so taking $k = 18$ pieces is optimal, and the answer is 8.

Sample 3

Large sample, not printed here; provided as the downloadable files `candy/candy3.in` and `candy/candy3.ans`.

Constraints

Test point	$n \leq$	$R \leq$	$R - L \leq$
1	2	5	5
2	5	10	10
3	10^3	10^3	10^3
4	10^5	10^5	10^5
5	10^3	10^9	0
6			10^3
7	10^5		10^5
8	10^9		10^9
9			
10			

In the original table the cell $n \leq 10^3$ spans test points 5–6, the cell $R \leq 10^9$ spans test points 5–10, and the cells $n \leq 10^9$ and $R - L \leq 10^9$ span test points 8–10.

For all data, it is guaranteed that $2 \leq n \leq L \leq R \leq 10^9$.

Problem 2: Insertion Sort (插入排序, sort) QOJ 18076

Time limit: 1 s. Memory limit: 512 MB. Total points: 100.

Description

Insertion sort is a very common and simple sorting algorithm. Little Z is a freshman, and today Teacher H has just taught the insertion sort algorithm in class.

Suppose comparing two elements takes $O(1)$ time; then insertion sort can sort an array of length n in $O(n^2)$ time. Suppose the n numbers are stored in $a_1, a_2, \cdots, a_n$; then the following pseudocode gives one of the simplest implementations of the insertion sort algorithm:

Below is the sample code in C/C++:

```
for (int i = 1; i <= n; i++)
    for (int j = i; j>=2; j--)
        if ( a[j] < a[j-1] ){
            int t = a[j-1];
            a[j-1] = a[j];
            a[j] = t;
        }
```

Below is the sample code in Pascal:

```
for i:=1 to n do
    for j:=i downto 2 do
        if a[j]<a[j-1] then
            begin
                t:=a[i];
                a[i]:=a[j];
                a[j]:=t;
            end;
```

To help Little Z understand insertion sort better, Little Z's teacher, Teacher H, assigned the following homework:

Teacher H gives an array a of length n, indexed from 1, in which all elements are non-negative integers. Little Z needs to support Q operations on the array a. There are two kinds of operations, with parameters as follows:

`1 x v`: this is the first kind of operation. It changes the x-th element of a, that is, the value of a_x, to v. It is guaranteed that $1 \le x \le n$ and $1 \le v \le 10^9$. Note that this operation changes the elements of the array; the modified array is kept and affects subsequent operations.

`2 x`: this is the second kind of operation. Suppose Teacher H sorts the array a according to the pseudocode above; you need to tell Teacher H the position, in the new sorted array, of the original x-th element of a, that is, a_x. It is guaranteed that $1 \le x \le n$. Note that this operation does not change the elements of the array; the sorted array is not kept and does not affect subsequent operations.

Teacher H does not like too many modifications, so he guarantees that the number of operations of type 1 does not exceed 5000.

Little Z has never studied for programming competitions, so Little Z cannot solve this problem. He has come to you to help him solve it.

Input

The first line of input contains two positive integers n, Q, denoting the length of the array and the number of operations. It is guaranteed that $1 \leq n \leq 8\,000$ and $1 \leq Q \leq 2 \times 10^5$.

The second line of input contains n space-separated non-negative integers, where the i-th non-negative integer denotes a_i. It is guaranteed that $1 \leq a_i \leq 10^9$.

The following Q lines each contain $2 \sim 3$ positive integers describing one operation; the format of the operations is given in the description.

Output

For every query of type 2, output one line containing one positive integer, the answer.

Sample 1 input

```
3 4
3 2 1
2 3
1 3 2
2 2
2 3
```

Sample 1 output

```
1
1
2
```

Explanation of sample 1

Before the modification operation, suppose Teacher H performs one insertion sort; then the positions of the three elements of the original sequence after the sort are $3, 2, 1$ respectively.

Before the modification operation, suppose Teacher H performs one insertion sort; then the positions of the three elements of the original sequence after the sort are $3, 1, 2$ respectively.

Note that although $a_2 = a_3$ at this moment, we cannot regard them as the same element.

Sample 2

Large sample, not printed here; provided as the downloadable files `sort/sort2.in` and `sort/sort2.ans`.

The data range of this test point is the same as that of test points $1 \sim 2$.

Sample 3

Large sample, not printed here; provided as the downloadable files `sort/sort3.in` and `sort/sort3.ans`.

The data range of this test point is the same as that of test points $3 \sim 7$.

Sample 4

Large sample, not printed here; provided as the downloadable files `sort/sort4.in` and `sort/sort4.ans`.

The data range of this test point is the same as that of test points $12 \sim 14$.

Constraints

For all test data, $1 \le n \le 8\,000$, $1 \le Q \le 2 \times 10^5$, $1 \le x \le n$, $1 \le v, a_i \le 10^9$.

For all test data, it is guaranteed that among all Q operations, at most 5000 operations are of type one.

The additional restrictions and scores of the test points are shown in the following table.

Test points	n	Q	Special property
1,2,3,4	≤ 10	≤ 10	None
5,6,7,8,9	≤ 300	≤ 300	
10,11,12,13	$\le 1\,500$	$\le 1\,500$	
14,15,16	$\le 8\,000$	$\le 8\,000$	All input a_i, v are guaranteed to be pairwise distinct
17,18,19			None
20,21,22		$\le 2 \times 10^5$	All input a_i, v are guaranteed to be pairwise distinct
23,24,25			None

In the original table the special property "None" spans test points 1–13, the cell $n \le 8\,000$ spans test points 14–25, the cell $Q \le 8\,000$ spans test points 14–19, and the cell $Q \le 2 \times 10^5$ spans test points 20–25.

Problem 3: Network Connection (网络连接, network) QOJ 18077

Time limit: 1 s. Memory limit: 512 MB. Total points: 100.

Description

The TCP/IP protocol is an important protocol in the field of network communication. Your task today is to try to use this protocol to reproduce a simplified network connection scenario.

In this problem, computers are divided into two major categories: servers (Server) and clients (Client). Servers are responsible for establishing connections, and clients are responsible for joining connections.

There are n computers in total that need to make network connections, numbered $1 \sim n$. These machines, in increasing order of their numbers, each issue one operation of establishing a connection or joining a connection.

When trying to establish or join a connection, each machine must provide an address string. The address string provided by a server denotes the address at which it tries to establish a connection; the address string provided by a client denotes the address of the connection it tries to join.

A well-formed address string must have the following features:

1. It must be of the form `a.b.c.d:e`, where a, b, c, d, e are all non-negative integers;
2. $0 \leq a, b, c, d \leq 255$ and $0 \leq e \leq 65535$;
3. none of a, b, c, d, e may contain redundant leading zeros.

Correspondingly, an address string that is not well-formed may have the following features:

1. It is not a string of the form `a.b.c.d:e`, for example it contains more than 3 characters `.` or more than 1 character `:`, etc.;
2. one or more of the integers a, b, c, d, e are outside the above ranges;
3. one or more of the integers a, b, c, d, e contain redundant leading zeros.

For example, the address string `192.168.0.255:80` is well-formed, but `192.168.0.999:80`, `192.168.00.1:10`, `192.168.0.1:088` and `192:168:0:1.233` are all not well-formed.

If the address string provided by a server or a client when issuing its operation is not well-formed, this operation is simply ignored.

In this problem, we assume that every address string satisfying the above rules can take part in a normal connection; you need not consider the actual meaning of each address string.

Due to network congestion and other reasons, two servers are not allowed to use the same address string; if this happens, the later server trying to establish a connection will fail to establish a connection. Apart from this, every server that provides a well-formed address string successfully establishes a connection.

If a client providing a well-formed address tries to join a connection and its address string is the same as the address string provided by some earlier server that has successfully established a connection, then this client successfully joins the connection, and we say that it is connected to this server; if no such server can be found, the client is considered to have failed to join a connection.

Please note that although two different servers are not allowed to use the same address string, several clients using the same address string, and the same server being connected to by several clients at the same time, are allowed.

Your task is simple: given the type and address string of every computer, determine the connection status of each computer.

Input

Line 1 contains one positive integer n.

The following n lines each contain 2 strings `op`, `ad`, giving the type and address string of each computer in increasing order of their numbers.

Here `op` is guaranteed to be one of the strings `Server` or `Client`, and `ad` is a non-empty string of length at most 25 consisting only of digits, the character `.` and the character `:`.

The two strings on each line are separated by exactly one space, and there are no extra spaces at the end of a line.

Output

Output n lines in total; line i contains one positive integer or one string denoting the connection status of computer i. Specifically:

If computer i is a server, then:

1. if it provides a well-formed address string and successfully establishes a connection, output the string `OK`;

2. if it provides a well-formed address string but fails to establish a connection because an earlier server has the same address string, output the string `FAIL`;

3. if the address string it provides is not well-formed, output the string `ERR`.

If computer i is a client, then:

1. if it provides a well-formed address string and successfully joins a connection, output one positive integer: the number of the server this client is connected to;

2. if it provides a well-formed address string but fails to join a connection, output the string `FAIL`;

3. if the address string it provides is not well-formed, output the string `ERR`.

Sample 1 input

```
5
Server 192.168.1.1:8080
Server 192.168.1.1:8080
Client 192.168.1.1:8080
Client 192.168.1.1:80
Client 192.168.1.1:99999
```

Sample 1 output

```
OK
FAIL
1
FAIL
ERR
```

Explanation of sample 1

Computer 1 is a server; it provides the well-formed address string `192.168.1.1:8080` and successfully establishes a connection;

computer 2 is a server; it provides the same address string as computer 1 and fails to establish a connection;

computer 3 is a client; it provides the well-formed address string `192.168.1.1:8080`, successfully joins a connection, and is connected to server 1;

computer 4 is a client; it provides the well-formed address string `192.168.1.1:80`, but no server can be found to connect to;

computer 5 is a client; the address string `192.168.1.1:99999` it provides is not well-formed.

Sample 2 input

```
10
Server 192.168.1.1:80
Client 192.168.1.1:80
Client 192.168.1.1:8080
Server 192.168.1.1:80
Server 192.168.1.1:8080
Server 192.168.1.999:0
Client 192.168.1.1.8080
Client 192.168.1.1:8080
Client 192.168.1.1:80
Client 192.168.1.999:0
```

Sample 2 output

```
OK
1
FAIL
FAIL
OK
ERR
ERR
5
1
ERR
```

Sample 3

Large sample, not printed here; provided as the downloadable files `network/network3.in` and `network/network3.ans`.

Sample 4

Large sample, not printed here; provided as the downloadable files `network/network4.in` and `network/network4.ans`.

Constraints

Test point numbers	$n \leq$	Special property
1	10	Properties 1, 2, 3
$2 \sim 3$	100	Properties 1, 2, 3
$4 \sim 5$	1000	
$6 \sim 8$		Properties 1, 2
$9 \sim 11$		Property 1
$12 \sim 13$		Property 2
$14 \sim 15$		Property 4
$16 \sim 17$		Property 5
$18 \sim 20$		No special property

In the original table the cell "Properties 1, 2, 3" spans test points 2–5, and the cell $n \leq 1000$ spans test points 4–20.

"Property 1" is: all address strings are guaranteed to be well-formed;

"Property 2" is: for any two different computers, if both are servers or both are clients, then the address strings they provide are guaranteed to be different;

"Property 3" is: the number of every server is guaranteed to be smaller than the numbers of all clients;

"Property 4" is: all address strings are guaranteed to be of the form `a.b.c.d:e`, where a, b, c, d, e are all non-negative integers not exceeding 10^9 and without redundant leading zeros;

"Property 5" is: all address strings are guaranteed to be of the form `a.b.c.d:e`, where a, b, c, d, e are all non-empty strings consisting only of digits.

For 100% of the data, it is guaranteed that $1 \leq n \leq 1000$.

Problem 4: Little Bear's Fruit Baskets (小熊的果篮, fruit) QOJ 18078

Time limit: 1 s. Memory limit: 512 MB. Total points: 100.

Description

In Little Bear's fruit shop there is a row of n fruits. Each fruit is either an apple or an orange, and the fruits are numbered from left to right with the positive integers $1, 2, 3, \ldots, n$. Fruits of the same kind that are arranged consecutively are called a "block". Little Bear wants to pick this row of fruits into several fruit baskets, in the following way: each time, the leftmost fruit of every "block" is picked out simultaneously, and together they form one fruit basket. This operation is repeated until the fruits are used up. Note that after each basket is picked, the "blocks" may change. For example, after the only orange between two apple "blocks" is picked away, the two apple "blocks" become one "block". Please help Little Bear compute the fruits contained in each fruit basket.

Input

The first line of input contains one positive integer n, denoting the number of fruits.

The second line of input contains n space-separated integers, where the i-th number denotes the kind of the fruit numbered i: 1 stands for an apple and 0 stands for an orange.

Output

Output several lines.

Line i describes the fruit basket formed by the fruits picked out in the i-th round. Output the numbers of all fruits in this basket in increasing order, with every two numbers separated by one space.

Sample 1 input

```
12
1 1 0 0 1 1 1 0 1 1 0 0
```

Sample 1 output

```
1 3 5 8 9 11
2 4 6 12
7
10
```

Explanation of sample 1

This is the explanation of the first set of sample data.

At the beginning the fruits are 1 1 0 0 1 1 1 0 1 1 0 0, forming 6 blocks in total.

In the first round of picking fruits to form a basket, the fruits numbered 1, 3, 5, 8, 9, 11 are picked out.

After that the remaining fruits are 1 0 1 1 1 0, forming 4 blocks in total.

In the second round of picking fruits to form a basket, the fruits numbered 2, 4, 6, 12 are picked out.

After that the remaining fruits are 1 1, forming only 1 block.

In the third round of picking fruits to form a basket, the fruit numbered 7 is picked out.
Finally the remaining fruit is 1, forming only 1 block.
In the fourth round of picking fruits to form a basket, the fruit numbered 10 is picked out.

Sample 2 input

```
20
1 1 1 1 0 0 0 1 1 1 0 0 1 0 1 1 0 0 0 0
```

Sample 2 output

```
1 5 8 11 13 14 15 17
2 6 9 12 16 18
3 7 10 19
4 20
```

Sample 3

Large sample, not printed here; provided as the downloadable files `fruit/fruit3.in` and `fruit/fruit3.ans`.

Constraints

For 10% of the data, $n \leq 5$.
For 30% of the data, $n \leq 1000$.
For 70% of the data, $n \leq 50000$.
For 100% of the data, $n \leq 2 \times 10^5$.

Note

Since the data size is large, C/C++ contestants are advised to use `scanf` and `printf` for input and output.

Source

CCF CSP-J 2021, second round (held 2021-10-23). Problem pages on QOJ, in contest order:

- `https://qoj.ac/problem/18075` (candy)
- `https://qoj.ac/problem/18076` (sort)
- `https://qoj.ac/problem/18077` (network)
- `https://qoj.ac/problem/18078` (fruit)

The statements were taken from the official PDF statement embedded on each QOJ problem page; the sample files were taken from the attachment archive of each page.